

LOW-LEVEL DESIGN

User Posts & Albums Viewer

Version	0.1
Status	Draft
Prepared By	Atul Sharma
Technology	HTML5, CSS3, JavaScript (ES6+), jQuery 4.0.0
Data Source	JSONPlaceholder REST API
Application Type	Client-side / presentation-layer web application

DOCUMENT PURPOSE Implementation-level design derived from the confirmed RUD v1.1 and corresponding HLD/SRS. The design remains intentionally lightweight because the assignment has no custom backend, database, authentication, or persistent storage.

1. Purpose

This Low-Level Design translates the confirmed requirements into an implementation-level design for the User Posts & Albums Viewer. It defines the client-side project structure, functions, DOM structure, API interactions, state handling, event handling, loading/error behavior, and rendering responsibilities.

The design is intentionally lightweight because the confirmed scope contains no custom backend, database, authentication, or persistent storage. JSONPlaceholder is the only external data source.

2. Requirements Baseline

The LLD is based on the confirmed requirements recorded in RUD v1.1 and the corresponding HLD/SRS.

ID	Confirmed requirement	LLD response
FR-01	Retrieve users from JSONPlaceholder	getUsers() + loadUsers()
FR-02	Display first name + last name	renderUsers() uses the API name display value only
FR-03	Allow user selection	Selectable user controls + delegated click handling
FR-04	Retrieve selected user posts	getUserPosts(userId)
FR-05	Retrieve selected user albums	getUserAlbums(userId)
FR-06	Posts and Albums are separate	activeView + renderPosts() / renderAlbums(); one shared content area
FR-07	Loading feedback	Separate users/details loading elements and state flags
FR-08	User-facing API error handling	Separate users/details error elements and failure handlers
FR-09	Update without full page refresh	jQuery DOM updates after requests complete

3. Scope and Design Boundaries

3.1 In scope

- Initial retrieval and rendering of the user list.
- Display of each user's first and last name as the requested display name.
- User selection.

- Retrieval of both posts and albums for the selected user.
- Separate Posts and Albums controls/views.
- Loading, error, and empty states.
- Client-side DOM rendering.
- Basic responsive/presentable CSS.

3.2 Out of scope

- Custom backend or server-side application code.
- Custom database or persistence.
- Authentication and authorization.
- User registration/login.
- CRUD operations against an application-owned data store.
- Advanced UI/UX beyond the confirmed feature structure.

4. Proposed Project Structure

```
// text
user-posts-albums-viewer/
|
+- index.html
+- css/
|   +- style.css
+- js/
|   +- api.js
|   +- app.js
+- docs/
|   +- RUD_User_Posts_Albums_v1.1.pdf
|   +- SRS_User_Posts_Albums_v0.1.pdf
|   +- API_Spec_User_Posts_Albums_v0.1.pdf
|   +- HLD_User_Posts_Albums_v0.1.pdf
|   +- LLD_User_Posts_Albums_v0.1.md
+- README.md
```

File	Responsibility
index.html	Static document structure and stable UI containers
css/style.css	Layout, responsive behavior, selected state, loading/error/empty styling
js/api.js	JSONPlaceholder request functions only
js/ui.js	DOM rendering and UI state updates only
js/app.js	State, event handling, orchestration, and request coordination
README.md	Setup, usage, and project overview

The separation is deliberately small; no framework or additional state-management library is introduced.

5. Runtime Flow

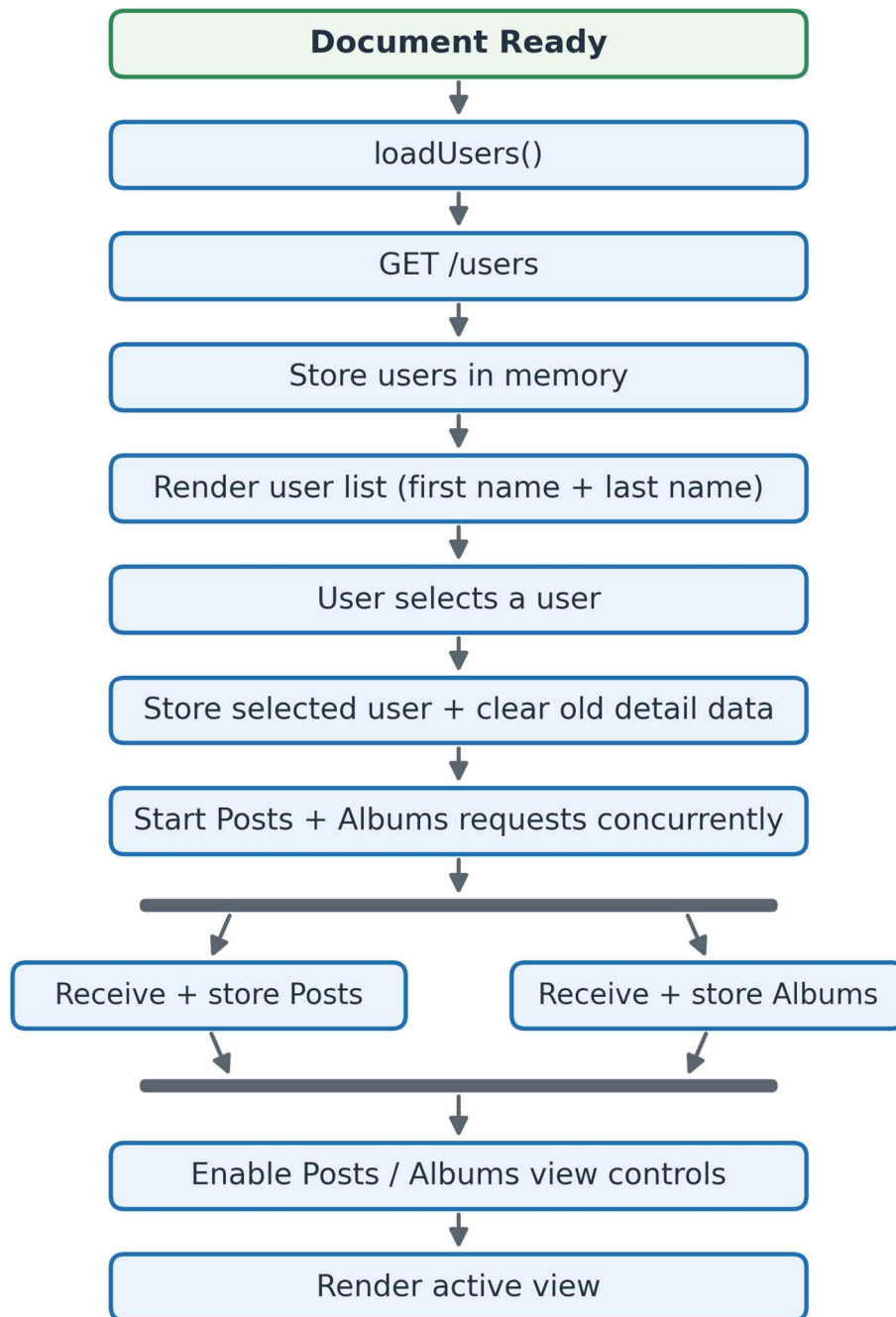


Figure 1. Low-level runtime flow

1. On document ready, loadUsers() starts the initial request.
2. The client requests GET /users.
3. Returned users are stored in memory and rendered using the name display value.
4. The user selects a user control.
5. The selected user is stored and previous user-specific data is cleared.
6. Posts and Albums requests start concurrently.
7. When both responses are available, the client stores both datasets and enables the separate view controls.
8. The active view is rendered into a single content area.

6. Client-Side State Model

The application keeps only session-local state in app.js.

```
// javascript
const state = {
  users: [],
  selectedUser: null,
  posts: [],
  albums: [],
  activeView: "posts",
  usersLoading: false,
  detailsLoading: false,
  usersError: null,
  detailsError: null,
  detailRequestId: 0
};
```

State field	Type	Responsibility
users	Array	Retrieved user records
selectedUser	Object / null	Current selected user
posts	Array	Posts for the selected user
albums	Array	Albums for the selected user
activeView	String	posts or albums
usersLoading	Boolean	Initial /users request state
detailsLoading	Boolean	Selected-user request state
usersError	String / null	Initial users error state
detailsError	String / null	Selected-user error state
detailRequestId	Number	Prevents stale responses from a previous selection from updating the current UI

activeView defaults to posts to match the sample UI state; the user can switch to albums after the selected user's detail data is available.

7. HTML / DOM Structure

Stable containers are defined in HTML and updated through jQuery.

```
// html
<body>
  <header>
    <h1>User Posts & Albums Viewer</h1>
  </header>

  <main>
    <section id="users-panel">
      <h2>Users</h2>
      <div id="users-loading" hidden></div>
      <div id="users-error" hidden></div>
      <div id="users-list"></div>
    </section>

    <section id="details-panel">
      <h2 id="selected-user"></h2>

      <div id="content-controls">
        <button id="posts-btn" type="button">Posts</button>
        <button id="albums-btn" type="button">Albums</button>
      </div>

      <div id="loading-message" hidden></div>
      <div id="error-message" hidden></div>
      <div id="content-area"></div>
    </section>
  </main>
</body>
```

Element	Responsibility
#users-list	Render selectable users
#users-loading	Show initial user-list loading state
#users-error	Show initial user-list error state
#selected-user	Show selected user's first and last name
#posts-btn	Activate Posts view
#albums-btn	Activate Albums view
#loading-message	Show selected-user loading state
#error-message	Show selected-user error state
#content-area	Display either Posts or Albums, never both together

8. API Layer (api.js)

api.js contains transport logic only. It must not modify the DOM.

8.1 Base URL

```
// javascript
const API_BASE_URL = "https://jsonplaceholder.typicode.com";
```

8.2 Retrieve users

```
// javascript
function getUsers() {
  return $.ajax({
    url: `${API_BASE_URL}/users`,
    method: "GET",
    dataType: "json"
  });
}
```

8.3 Retrieve posts

```
// javascript
function getUserPosts(userId) {
  return $.ajax({
    url: `${API_BASE_URL}/users/${userId}/posts`,
    method: "GET",
    dataType: "json"
  });
}
```

8.4 Retrieve albums

```
// javascript
function getUserAlbums(userId) {
  return $.ajax({
    url: `${API_BASE_URL}/users/${userId}/albums`,
    method: "GET",
    dataType: "json"
  });
}
```

API-LAYER RULE The API layer returns the result of the jQuery AJAX operation. UI rendering, loading messages, and error presentation are handled by app.js and ui.js.

9. Application Logic (app.js)

app.js owns state, event handling, request orchestration, and selection logic.

9.1 Initialization

```
// javascript
$(function () {
  loadUsers();
  bindEvents();
});
```

9.2 Initial user retrieval

The initial request has its own loading and error state so the users area can communicate the request status independently of selected-user details.

```
// javascript
function loadUsers() {
  state.usersLoading = true;
  state.usersError = null;

  showUsersLoading();
  clearUsersError();

  return getUsers()
    .done(function (users) {
      state.users = users;
      renderUsers(state.users);
    })
    .fail(function () {
      state.usersError = "Failed to fetch users.";
      showUsersError(state.usersError);
    })
    .always(function () {
      state.usersLoading = false;
      hideUsersLoading();
    });
}
```

9.3 User selection

```
// javascript
function handleUserSelection(user) {
  if (!user || !user.id) {
    showError("Unable to identify the selected user.");
    return;
  }

  state.selectedUser = user;
  state.posts = [];
  state.albums = [];
  state.activeView = "posts";
  state.detailsError = null;
  state.detailRequestId += 1;

  $("#selected-user").text(user.name);
  clearContentArea();
  clearError();

  loadUserDetails(user.id, state.detailRequestId);
}
```

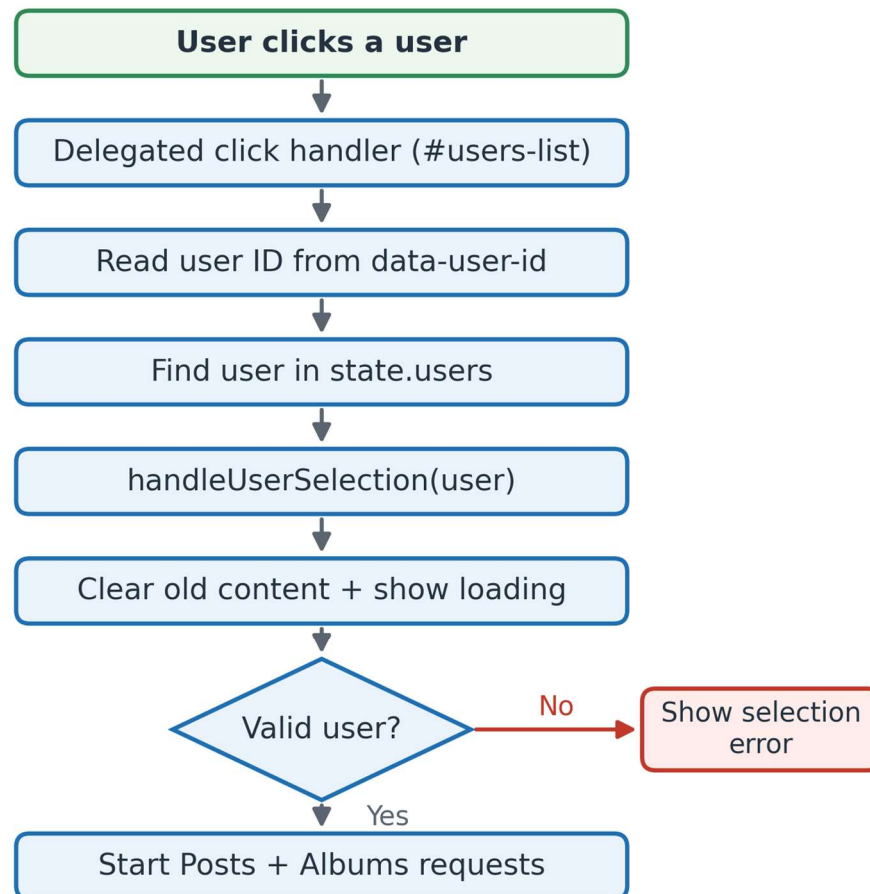


Figure 2. Low-level user-selection sequence

9.4 Loading posts and albums

The two resources are independent, so they are initiated concurrently with `$.when()`.

```
// javascript
function loadUserDetails(userId, requestId) {
    state.detailsLoading = true;
    showLoading();
    clearError();
    setContentControlsEnabled(false);

    return $.when(
        getUserPosts(userId),
        getUserAlbums(userId)
    )
    .done(function (posts, albums) {
        if (requestId !== state.detailRequestId) {
            return;
        }

        state.posts = posts[0];
        state.albums = albums[0];
        state.detailsError = null;

        setContentControlsEnabled(true);
        renderActiveView();
    })
    .fail(function () {
        if (requestId !== state.detailRequestId) {
            return;
        }

        state.detailsError = "Failed to load user details.";
        clearContentArea();
        showError(state.detailsError);
    })
    .always(function () {
        if (requestId === state.detailRequestId) {
            state.detailsLoading = false;
            hideLoading();
        }
    });
}
```

The request ID prevents an earlier selection from overwriting the UI after the user has already selected another user.

9.5 View switching

```
// javascript
function handleViewChange(view) {
    if (state.detailsLoading || state.detailsError) {
        return;
    }

    state.activeView = view;
    clearError();
    renderActiveView();
}

function renderActiveView() {
    if (state.activeView === "posts") {
        renderPosts(state.posts);
    } else {
        renderAlbums(state.albums);
    }
}
```

10. UI Rendering Layer (ui.js)

ui.js performs DOM updates and contains no API transport logic.

10.1 Render users

JSONPlaceholder exposes a single name field for each user. For this assignment, that full display value is treated as the required first-name + last-name presentation. No additional name-parsing rule is introduced.

```
// javascript
function renderUsers(users) {
  const $list = $("#users-list");
  $list.empty();

  $.each(users, function (_, user) {
    const $button = $("", {
      type: "button",
      class: "user-item",
      text: user.name,
      "data-user-id": user.id
    });

    $list.append($button);
  });
}
```

10.2 Render Posts

```
// javascript
function renderPosts(posts) {
  const $content = $("#content-area");
  $content.empty();

  const $heading = $("

### ").text(`Posts: ${posts.length}`); $content.append($heading); if (posts.length === 0) { $content.append($("<p>").text("No posts available for this user.")).return; } const $list = $("<ul>"); $.each(posts, function (_, post) { const $item = $("<li>"); $("<strong>").text(post.title).appendTo($item); $("<p>").text(post.body).appendTo($item); $item.appendTo($list); }); $content.append($list); }


```

10.3 Render Albums

```
// javascript
function renderAlbums(albums) {
  const $content = $("#content-area");
  $content.empty();

  const $heading = $("

### ").text(`Albums: ${albums.length}`); $content.append($heading); if (albums.length === 0) { $content.append($("<p>").text("No albums available for this user.")); return; } const $list = $("<ul>"); $.each(albums, function (_, album) { $("<li>") .text(album.title) .appendTo($list); }); $content.append($list); }


```

10.4 Loading and error feedback

```
// javascript
function showUsersLoading() {
    $("#users-loading")
        .text("Loading users...")
        .prop("hidden", false);
}

function hideUsersLoading() {
    $("#users-loading").prop("hidden", true);
}

function showLoading() {
    $("#loading-message")
        .text("Loading posts and albums...")
        .prop("hidden", false);
}

function hideLoading() {
    $("#loading-message").prop("hidden", true);
}

function showUsersError(message) {
    $("#users-error")
        .text(message)
        .prop("hidden", false);
}

function showError(message) {
    $("#error-message")
        .text(message)
        .prop("hidden", false);
}

function clearUsersError() {
    $("#users-error").prop("hidden", true).empty();
}

function clearError() {
    $("#error-message").prop("hidden", true).empty();
}

function clearContentArea() {
    $("#content-area").empty();
}

function setContentControlsEnabled(enabled) {
    $("#posts-btn, #albums-btn").prop("disabled", !enabled);
}
```

11. Event Handling

Because the user controls are created dynamically, delegated event handling is used on the stable #users-list container.

```
// javascript
function bindEvents() {
  $("#users-list").on("click", ".user-item", function () {
    const userId = Number($(this).data("user-id"));
    const user = state.users.find(function (item) {
      return item.id === userId;
    });

    handleUserSelection(user);
  });

  $("#posts-btn").on("click", function () {
    handleViewChange("posts");
  });

  $("#albums-btn").on("click", function () {
    handleViewChange("albums");
  });
}
```

EVENT FLOW User clicks user -> delegated click handler -> obtain user ID -> find user in state -> handleUserSelection(user) -> request posts + albums.

12. Detailed User-Selection Sequence

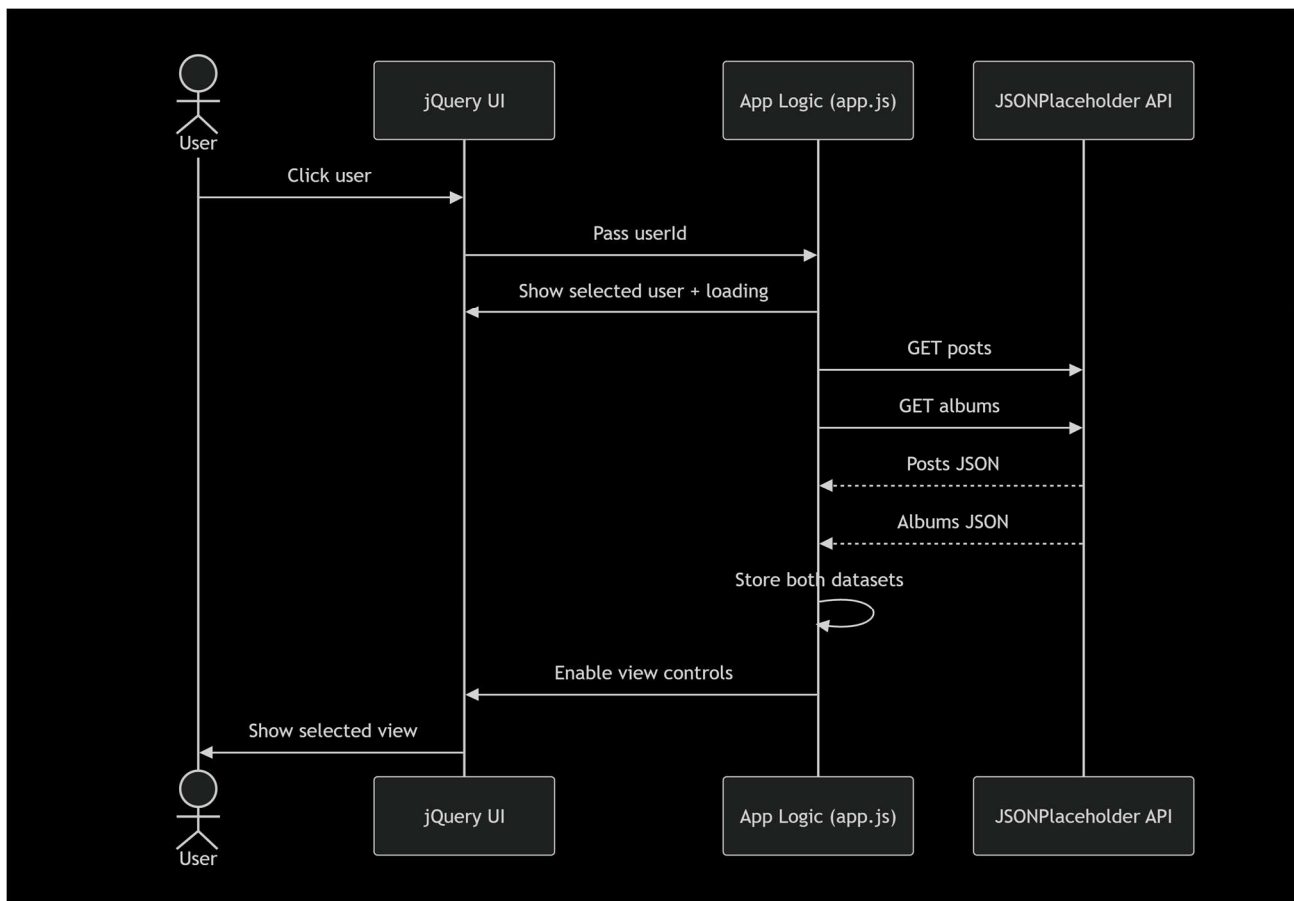


Figure 3. Detailed user-selection sequence

The selected user is updated immediately, the old content is cleared, and the two user-specific API requests run independently of each other.

13. Posts / Albums View Logic

The selected-user panel contains two separate controls:

```
// ui
[ Posts ]    [ Albums ]
```

The content area is shared, but only one view is rendered at any time.

Posts view

```
// flow
activeView = "posts"
-> renderPosts(state.posts)
```

Albums view

```
// flow
activeView = "albums"
-> renderAlbums(state.albums)
```

VIEW-SWITCH BEHAVIOR Switching views after the two datasets have been loaded does not create another API request. It only changes the rendered content.

14. Loading, Error, and Empty-State Handling

14.1 Initial users loading

```
// flow
Application start
-> Loading users...
-> GET /users
-> success -> render user list
-> failure -> show users error
```

14.2 Selected-user loading

```
// flow
User selected
-> clear old detail content
-> Loading posts and albums...
-> GET posts + GET albums
-> both succeed -> enable controls + render active view
-> failure -> hide loading + show error
```

14.3 Empty data

A successful response with an empty array should not produce a blank content area. The renderer displays a clear message such as:

```
// text
No posts available for this user.

or:

No albums available for this user.
```

14.4 Stale-response protection

When the user selects another user before the previous requests finish, `detailRequestId` ensures that an older result is ignored. This prevents the previously selected user's posts or albums from being rendered under the newly selected user's name.

15. Data Shapes Used by the UI

Only fields required by the interface are consumed.

User

```
// javascript
{
  id: 1,
  name: "Leanne Graham",
  // other API fields ignored by this UI
}
```

Used fields: id, name

Post

```
// javascript
{
  userId: 1,
  id: 1,
  title: "...",
  body: "..."
}
```

Used fields: id, title, body

Album

```
// javascript
{
  userId: 1,
  id: 1,
  title: "..."
}
```

Used fields: id, title

16. Request and Concurrency Behavior

Posts and Albums are independent resources. The client therefore starts both requests without waiting for one request to complete before starting the other.

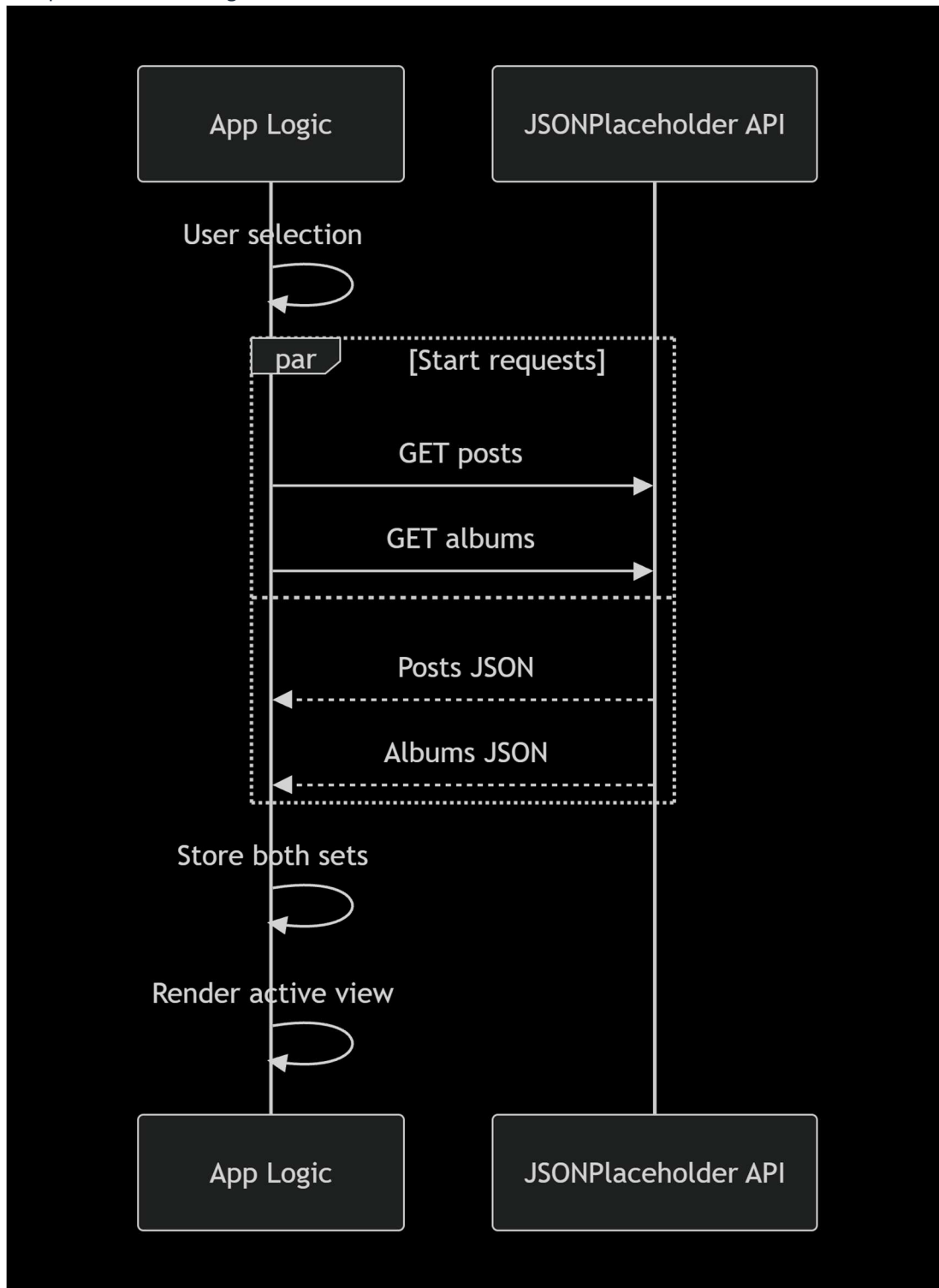


Figure 4. Request and concurrency behavior

The currently selected user's data is kept only in memory. No persistent cache is required.

17. Security Considerations

- API-provided text is treated as untrusted input for rendering.
- jQuery .text() is preferred for API-provided values rather than injecting raw HTML strings.
- No API credentials, tokens, or authentication secrets are required for this assignment.
- No authentication or authorization layer is included because it is outside the confirmed scope.

18. Maintainability and Design Rules

- Keep API communication in api.js.
- Keep DOM rendering in ui.js.
- Keep state and workflow orchestration in app.js.
- Use stable IDs/classes for DOM targeting.
- Clear or replace the current content before rendering a new dataset.
- Use the selected user's ID to construct user-specific endpoint URLs.
- Do not combine Posts and Albums into one view.
- Avoid unnecessary abstraction because the assignment has a small scope.

19. Requirement Traceability

RUD/SRS requirement	LLD implementation
Retrieve users	getUsers() + loadUsers()
Display first + last name	renderUsers() using user.name
Select user	Delegated click handler + handleUserSelection()
Retrieve posts	getUserPosts(userId)
Retrieve albums	getUserAlbums(userId)
Separate Posts/Albums	activeView, renderPosts(), renderAlbums()
Loading feedback	showUsersLoading(), showLoading() and corresponding hide functions
Error handling	Separate users/details error handlers
Dynamic update	jQuery DOM updates without full page reload

20. Implementation Checklist

- ☐ jQuery 4.0.0 is loaded.
- ☐ GET /users works on document ready.
- ☐ Users display first name + last name only.
- ☐ User controls are selectable.
- ☐ Selected user name is shown.
- ☐ Posts and Albums requests use the selected user ID.
- ☐ Posts and Albums requests can run concurrently.
- ☐ Posts and Albums are separate views.
- ☐ Switching views does not combine the two datasets.
- ☐ Loading feedback is visible during retrieval.
- ☐ Initial users errors are shown in the users area.
- ☐ Selected-user errors are shown in the details area.

- [] Empty arrays produce meaningful empty states.
- [] Selecting another user replaces previous detail data.
- [] Stale responses from older selections are ignored.
- [] No full-page refresh is required for user selection.
- [] API-provided text is rendered safely.
- [] No console errors remain during normal use.

21. Design Decisions and HLD/LLD Boundary

- jQuery is used for AJAX, event handling, and DOM manipulation.
- API functions are separated from UI rendering functions.
- Posts and Albums are requested concurrently after user selection.
- Both datasets are stored in memory for the current selection so view switching does not require another API request.
- A request ID prevents stale responses from changing the UI after a new user is selected.
- Posts and Albums use separate views in a shared content area.

The HLD remains responsible for the major component architecture and system boundaries. Exact selectors, function names, state fields, and jQuery operations are intentionally kept in this LLD.

22. Document Status

LLD V0.1 - DRAFT This document incorporates the confirmed RUD requirements, including first-name + last-name display, separate Posts and Albums views, loading/error states, and the client-side-only architecture. It also resolves low-level consistency issues by separating initial/detail loading and error states, treating the API name field as the display name, and protecting against stale user-detail responses.